

# Basic setup and installation

## 1 Open a shell

### 1.1 Windows

### 1.2 macOS

### 1.3 Linux

## 2 Install Julia

### 2.1 Windows

Install Julia from the [Windows store](https://www.microsoft.com/store/apps/9NJNWW8PVKMN), found here:

<https://www.microsoft.com/store/apps/9NJNWW8PVKMN>

or using the following command in the shell:

```
winget install julia -s msstore in a terminal
```

### 2.2 Linux and macOS

Install Julia by entering the following command in your shell.

```
curl -fsSL https://install.julialang.org | sh
```

## 3 Install the required packages

### 3.1 Activate the Julia environment

If installing into the global environment, start the global Julia environment in your shell:

```
julia
```

If installing into a project environment, navigate to the project directory first in your shell and then start a Julia REPL session using the `--project` flag:

```
cd ~/projects/paleodb  
julia --project=.
```

### 3.2 Import the package manager and confirm environment status

Either way, confirm that you are running in the correct environment by using the `status` function of the Julia package manager, `Pkg`:

```
using Pkg  
Pkg.status()
```

A typical output if the global environment is active might be:

```
Status `~/\.julia/environments/v1.12/Project.toml`  
 [5fb14364] OhMyREPL v0.5.32  
 [295af30f] Revise v3.11.0  
Info Packages marked with   have new versions available and may be upgradable.
```

The first line of output from the `Pkg.status()` function is the path to the active Julia project file, and here, the path is to the global Julia environment.

```
Status `~/\.julia/environments/v1.12/Project.toml`
```

Global installs are convenient, with packages available to any Julia program you run. Some workflows benefit from a “heavy” global environment with most common packages installed there `DataFrames.jl`, `CSV.jl`, `XLSX.jl`, `Makie.jl`, `Distributions.jl` and universally available to every project, with project-specific installs limited to specialized needs.

On the other hand, the lack of isolation between global and project-specific environments can result in globally-installed packages leaking into project tests, dependency chains, and so on. Workflows incorporating software engineering best practices typically benefit from having light global environments and heavy project environments: little to no packages installed globally, with all packages installed into project specific environments.

If a project environment is active, on the other hand, will hand you would see the path to the project directory.

```
Status `~/projects/learning/juliacamp/paleobiology`
```

### 3.3 Install the basic packages

```
Pkg.add([  
    "PaleobiologyDB",  
    "DataFrames",  
    "Statistics",  
    "CairoMakie",  
])
```